

LAB ACTIVITY (Lab-07)

1. Bank Account System

- Create an **abstract class** `Account` with common fields: `accountNumber`, `balance`.
- Add an abstract method `withdraw()`.
- Implement constructors in the subclasses.
- Create subclasses: `SavingsAccount`, `CurrentAccount` that implement `withdraw()` differently.
- Create an **interface** `Depositable` with method `deposit()` → implement it in both subclasses.
- Show **method overloading** in `deposit()` (`deposit(int amount)`, `deposit(double amount)`).

```
interface Depositable
{
    void deposit(int amount);
    void deposit(double amount);
}
abstract class Account
{
    int accountNumber;
    double balance;
    Account(int accountNumber, double balance)
    {
        this.accountNumber= accountNumber;
        this.balance= balance;
    }
    abstract void withdraw(double amount);
    void showBalance()
    {
        System.out.println("Account No.:"+accountNumber+",Balance:"+balance);
    }
}
class SavingsAccount extends Account implements Depositable
{
    private static final double MIN_BALANCE=500.0;
    SavingsAccount(int accountNumber, double balance)
    {
        super(accountNumber, balance);
    }
    void withdraw(double amount)
    {
        if(balance-amount>=MIN_BALANCE)
        {
            balance-=amount;
            System.out.println("SavingsAccount:withdrawn"+ amount);
        }
        else
        {
            System.out.println("SavingsAccount:withdrawl denied! Minimum balance must be mantained");
        }
    }
    public void deposit(int amount)
    {
        balance+=amount;
        System.out.println("SavingsAccount: DEposited"+ amount);
    }
}
```

```

    }
    public void deposit(double amount)
    {
        balance+=amount;
        System.out.println("SavingsAccount: DEposited"+ amount);
    }
}

class CurrentAccount extends Account implements Depositable
{
    private static final double OVERDRAFT_LIMIT=1000.0;
    CurrentAccount(int accountNumber, double balance)
    {
        super(accountNumber, balance);
    }
    void withdraw(double amount)
    {
        if(balance-amount>=OVERDRAFT_LIMIT)
        {
            balance-=amount;
            System.out.println("CurrentAccount:withdrawn"+ amount);
        }
        else
        {
            System.out.println("CurrentAccount:withdrawl denied! Minimum balance must be mantained");
        }
    }
    public void deposit(int amount)
    {
        balance+=amount;
        System.out.println("CurrentAccount: DEposited"+ amount);
    }
    public void deposit(double amount)
    {
        balance+=amount;
        System.out.println("CurrentAccount: DEposited"+ amount);
    }
}

class bank
{
    public static void main(String[] ar)
    {
        SavingsAccount sa=new SavingsAccount(1234, 3000.0);
        CurrentAccount ca =new CurrentAccount(5678, 2000.0);
        sa.showBalance();
        sa.deposit(600);
        sa.withdraw(3200);
        sa.showBalance();
        System.out.println("-----");
        ca.showBalance();
        ca.deposit(600);
        ca.withdraw(1200);
        ca.showBalance();
    }
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab7(2009)> cd "d:\SEMESTER I\LAB WORK\
if ($?) { java bank }
Account No.:1234,Balance:3000.0
SavingsAccount: DEposited600
SavingsAccount:withdrawl denied! Minimum balance must be mantained
Account No.:1234,Balance:3600.0
-----
Account No.:5678,Balance:2000.0
CurrentAccount: DEposited600
CurrentAccount:withdrawn1200.0
Account No.:5678,Balance:1400.0
PS D:\SEMESTER I\LAB WORK\Java Programming\lab7(2009)> 

```

2. Vehicle Hierarchy

- Create an **abstract class** `Vehicle` with fields `speed` and `fuel`, and an abstract method `mileage()`.
- Subclasses: `Car`, `Bike`, `Truck` → override `mileage()`.
- Create an **interface** `Acceleratable` with `accelerate()` method.
- Show **method overloading** in `accelerate()` (`accelerate(int increment)`, `accelerate(int increment, int gear)`).
- Subclasses implement both abstract methods and interface methods.

```

interface Acceleratable {
    void accelerate(int increment);
    void accelerate(int increment, int gear);
}

abstract class Vehicle {
    double speed;
    double fuel;

    Vehicle(double speed, double fuel) {
        this.speed = speed;
        this.fuel = fuel;
    }

    abstract void mileage();

    public void display() {
        System.out.println("Speed: " + speed + " km/h, Fuel: " + fuel + " liters");
    }
}

class Car extends Vehicle implements Acceleratable {
    public Car(double speed, double fuel) {
        super(speed, fuel);
    }

    @Override
    public void mileage() {
        double mileage = fuel * 15;
        System.out.println("Car mileage: " + mileage + " km");
    }

    public void accelerate(int increment) {

```

```

        speed += increment;
        System.out.println("New Speed: " + speed);
    }

    public void accelerate(int increment, int gear) {
        speed += increment+gear * 10;
        System.out.println("New Speed(gear): " + speed);
    }
}

class Bike extends Vehicle implements Acceleratable {
    public Bike(double speed, double fuel) {
        super(speed, fuel);
    }

    @Override
    public void mileage() {
        double mileage = fuel * 40;
        System.out.println("Bike mileage: " + mileage + " km");
    }

    public void accelerate(int increment) {
        speed += increment;
        System.out.println("New Speed: " + speed);
    }

    public void accelerate(int increment, int gear) {
        speed += increment+ gear * 10;
        System.out.println("New Speed(gear): " + speed);
    }
}

class Truck extends Vehicle implements Acceleratable {
    public Truck(double speed, double fuel) {
        super(speed, fuel);
    }

    @Override
    public void mileage() {
        double mileage = fuel * 8;
        System.out.println("Truck mileage: " + mileage + " km");
    }

    public void accelerate(int increment) {
        speed += increment;
        System.out.println("New Speed: " + speed);
    }

    public void accelerate(int increment, int gear) {
        speed += increment+ gear * 10;
        System.out.println("New Speed(gear): " + speed);
    }
}

public class MainVehicle {
    public static void main(String[] args) {
        // Car
        System.out.println("-----Car -----");
        Car car = new Car(60, 20);
    }
}

```

```

car.display();
car.accelerate(2);
car.accelerate(2, 2);
car.mileage();

System.out.println("-----Bike-----");

// Bike
Bike bike = new Bike(40, 5);
bike.display();
bike.accelerate(3, 4);
bike.accelerate(3);
bike.mileage();

System.out.println("-----Truck-----");

// Truck
Truck truck = new Truck(50, 50);
truck.display();
truck.accelerate(4);
truck.accelerate(4, 2);
truck.mileage();
}
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab7(2009)> cd "d:\SEM
ava } ; if ($?) { java MainVehicle }
-----Car -----
Speed: 60.0 km/h, Fuel: 20.0 liters
New Speed: 62.0
New Speed(gear): 84.0
Car mileage: 300.0 km
-----Bike-----
Speed: 40.0 km/h, Fuel: 5.0 liters
New Speed(gear): 83.0
New Speed: 86.0
Bike mileage: 200.0 km
-----Truck-----
Speed: 50.0 km/h, Fuel: 50.0 liters
New Speed: 54.0
New Speed(gear): 78.0
Truck mileage: 400.0 km

```

3. Shape Area Calculator

- Create an **abstract class** `Shape` with abstract method `calculateArea()`.
- Subclasses: `Circle`, `Rectangle`, `Triangle` override `calculateArea()`.
- In `Rectangle`, implement **method overloading** for area calculation (square vs rectangle).
- Create an **interface** `Drawable` with `draw()` method. Each shape implements it.

```
interface Drawable {
    void draw();
}

abstract class Shape implements Drawable {
    abstract double calculateArea();
}

class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a Circle with radius " + radius);
    }
}

class Rectangle extends Shape {
    double length;
    double width;

    // Rectangle constructor
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    // Square constructor (overloading)
    Rectangle(double side) {
        this.length = side;
        this.width = side;
    }

    @Override
    double calculateArea() {
        return length * width;
    }

    @Override
    public void draw() {
```

```

        if (length == width) {
            System.out.println("Drawing a Square with side " + length);
        } else {
            System.out.println("Drawing a Rectangle " + length + " x " + width);
        }
    }
}

class Triangle extends Shape {
    double base;
    double height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    double calculateArea() {
        return 0.5 * base * height;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a Triangle with base " + base + " and height " + height);
    }
}

public class MainShape {
    public static void main(String[] args) {
        // Circle
        Circle circle = new Circle(5);
        circle.draw();
        System.out.println("Circle area: " + circle.calculateArea());
        System.out.println("-----");

        // Rectangle
        Rectangle rectangle = new Rectangle(10, 5);
        rectangle.draw();
        System.out.println("Rectangle area: " + rectangle.calculateArea());

        // Square using overloaded constructor
        Rectangle square = new Rectangle(6);
        square.draw();
        System.out.println("Square area: " + square.calculateArea());
        System.out.println("-----");

        // Triangle
        Triangle triangle = new Triangle(8, 4);
        triangle.draw();
        System.out.println("Triangle area: " + triangle.calculateArea());
    }
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab7(2009)> cd "d:\SEMEST
a } ; if ($?) { java MainShape }
Drawing a Circle with radius 5.0
Circle area: 78.53981633974483
-----
Drawing a Rectangle 10.0 x 5.0
Rectangle area: 50.0
Drawing a Square with side 6.0
Square area: 36.0
-----
Drawing a Triangle with base 8.0 and height 4.0
Triangle area: 16.0

```

4. Employee Payroll System

- Create an **abstract class** `Employee` with abstract method `calculateBonus()`.
- Subclasses: `Manager`, `Developer`, `Intern` implement it differently.
- Add constructors in each subclass.
- Create an **interface** `Payable` with `calculateSalary()` method.
- Show **method overloading** for `calculateBonus()` (bonus by % OR by fixed amount).

```

interface Payable {
    double calculateSalary();
}

```

```

abstract class Employee {
    String name;
    double basicSalary;
    String post;

    Employee(String name, double basicSalary, String post) {
        this.name = name;
        this.basicSalary = basicSalary;
        this.post = post;
    }

    abstract double calculateBonus(double percent);
    abstract double calculateBonus(int fixedAmount);
}

```

```

class Manager extends Employee implements Payable {
    Manager(String name, double basicSalary) {
        super(name, basicSalary, "Manager");
    }

    @Override
    double calculateBonus(double percent) {
        double bonus = basicSalary * (percent / 100);
        System.out.println(post + " " + name + " Bonus (" + percent + "%): " + bonus);
        return bonus;
    }
}

```

```

@Override
double calculateBonus(int fixedAmount) {
    System.out.println(post + " " + name + " Bonus (fixed): " + fixedAmount);
    return fixedAmount;
}

@Override
public double calculateSalary() {
    return basicSalary;
}
}

class Developer extends Employee implements Payable {
    Developer(String name, double basicSalary) {
        super(name, basicSalary, "Developer");
    }

    @Override
    double calculateBonus(double percent) {
        double bonus = basicSalary * (percent / 100);
        System.out.println(post + " " + name + " Bonus (" + percent + "%): " + bonus);
        return bonus;
    }

    @Override
    double calculateBonus(int fixedAmount) {
        System.out.println(post + " " + name + " Bonus (fixed): " + fixedAmount);
        return fixedAmount;
    }

    @Override
    public double calculateSalary() {
        return basicSalary;
    }
}

class Intern extends Employee implements Payable {
    Intern(String name, double basicSalary) {
        super(name, basicSalary, "Intern");
    }

    @Override
    double calculateBonus(double percent) {
        double bonus = basicSalary * (percent / 100);
        System.out.println(post + " " + name + " Bonus (" + percent + "%): " + bonus);
        return bonus;
    }

    @Override
    double calculateBonus(int fixedAmount) {
        System.out.println(post + " " + name + " Bonus (fixed): " + fixedAmount);
        return fixedAmount;
    }

    @Override
    public double calculateSalary() {
        return basicSalary;
    }
}

```

```

public class PayrollSystem {
    public static void main(String[] args) {
        Manager m = new Manager("Alice", 80000);
        Developer d = new Developer("Bob", 50000);
        Intern i = new Intern("Charlie", 20000);

        System.out.println(m.post + " " + m.name + " Base Salary: " + m.calculateSalary());
        m.calculateBonus(20.0);
        m.calculateBonus(10000);

        System.out.println("-----");

        System.out.println(d.post + " " + d.name + " Base Salary: " + d.calculateSalary());
        d.calculateBonus(10.0);
        d.calculateBonus(5000);

        System.out.println("-----");

        System.out.println(i.post + " " + i.name + " Base Salary: " + i.calculateSalary());
        i.calculateBonus(5.0);
        i.calculateBonus(2000);
    }
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab7(2009)> cd "d:\SEMEST
.java } ; if ($?) { java PayrollSystem }
Manager Alice Base Salary: 80000.0
Manager Alice Bonus (20.0%): 16000.0
Manager Alice Bonus (fixed): 10000
-----
Developer Bob Base Salary: 50000.0
Developer Bob Bonus (10.0%): 5000.0
Developer Bob Bonus (fixed): 5000
-----
Intern Charlie Base Salary: 20000.0
Intern Charlie Bonus (5.0%): 1000.0
Intern Charlie Bonus (fixed): 2000

```

5. Library Management System

- Create an **abstract class** `Book` with abstract method `getFinalPrice()`.
- Subclasses: `EBook`, `PrintedBook` implement it differently.
- Create an **interface** `Discountable` with method `applyDiscount()`.
- Implement **method overloading** for `applyDiscount()` (percentage OR fixed).
- Constructors should be used to initialize `title`, `author`, `price`.

```
interface Discountable {
    void applyDiscount(double percent);
    void applyDiscount(int amount);
}

abstract class Book {
    String title;
    String author;
    double price;

    Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }

    abstract double getFinalPrice();
}

class EBook extends Book implements Discountable {
    double downloadFee = 50;

    EBook(String title, String author, double price) {
        super(title, author, price);
    }

    @Override
    public void applyDiscount(double percent) {
        price -= price * (percent / 100);
    }

    @Override
    public void applyDiscount(int amount) {
        price -= amount;
    }

    @Override
    double getFinalPrice() {
        return price + downloadFee;
    }
}

class PrintedBook extends Book implements Discountable {
    double shippingCost = 100;

    PrintedBook(String title, String author, double price) {
        super(title, author, price);
    }
}
```

```

@Override
public void applyDiscount(double percent) {
    price -= price * (percent / 100);
}

@Override
public void applyDiscount(int amount) {
    price -= amount;
}

@Override
double getFinalPrice() {
    return price + shippingCost;
}
}

public class LibrarySystem {
    public static void main(String[] args) {
        EBook ebook = new EBook("Digital Java", "Alice", 500);
        PrintedBook printed = new PrintedBook("Java Basics", "Bob", 800);

        System.out.println("EBook: " + ebook.title + " by " + ebook.author);
        ebook.applyDiscount(10.0);
        System.out.println("Final Price after discount: " + ebook.getFinalPrice());

        System.out.println("-----");

        System.out.println("PrintedBook: " + printed.title + " by " + printed.author);
        printed.applyDiscount(200);
        System.out.println("Final Price after discount: " + printed.getFinalPrice());
    }
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab7(2009)> cd "d
.java } ; if ($?) { java LibrarySystem }
EBook: Digital Java by Alice
Final Price after discount: 500.0
-----
PrintedBook: Java Basics by Bob
Final Price after discount: 700.0

```